NAME: R. Reddy Balaji

Reg: 192424122

Course:DSA0602-Data Handling and Data visualization

Experiment 18: Product Inventory Management

Aim

To analyse product inventory data using Python and R by creating a bar chart, stacked bar chart, and inventory table for effective visualization of product quantities.

Procedure

1. Create the product inventory dataset.
2. Load the required libraries.
3. Create a bar chart to display the quantity available for each product.
4. Create a stacked bar chart to display product quantities by category.
5. Display the inventory data in a table.
6. Analyse the charts and table.
7. Interpret the results.

Algorithm

1. Start.
2. Create a dataset containing Product ID, Product Name, Quantity Available, Price, and Category.
3. Read the dataset into the program.
4. Plot a bar chart showing product quantities.
5. Plot a stacked bar chart based on product categories.
6. Display the inventory table.

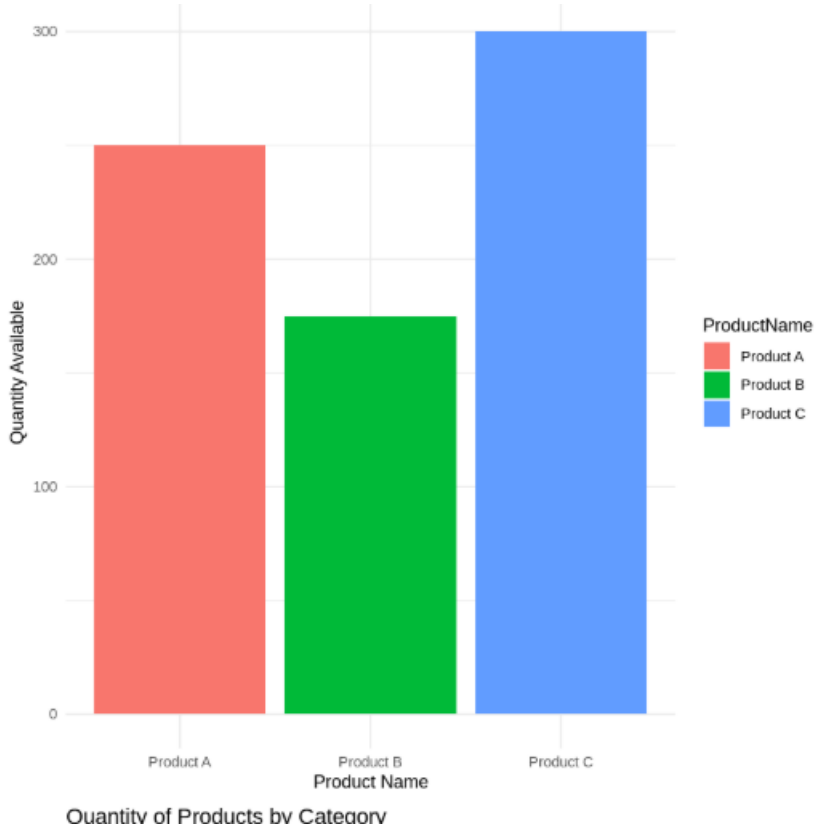
7. Stop.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt

# Dataset
data = {
    "Product ID":[1,2,3],
    "Product Name":["Product A","Product B","Product C"],
    "Quantity Available":[250,175,300],
    "Price":[20,15,18],
    "Category":["Electronics","Electronics","Furniture"]
}

df = pd.DataFrame(data)

# Display Table
print(df)

# -------------------------
# Bar Chart
# -------------------------
plt.figure(figsize=(6,4))
plt.bar(df["Product Name"], df["Quantity Available"],
color=["blue","green","orange"])
plt.title("Quantity Available for Each Product")
```

```
plt.xlabel("Product Name")
plt.ylabel("Quantity Available")
plt.show()

# -------------------------
# Stacked Bar Chart
# -------------------------
pivot = df.pivot_table(index="Category",
                       columns="Product Name",
                       values="Quantity Available",
                       aggfunc="sum").fillna(0)

pivot.plot(kind="bar", stacked=True, figsize=(6,5))
plt.title("Product Quantity by Category")
plt.xlabel("Category")
plt.ylabel("Quantity Available")
plt.show()
```

output:



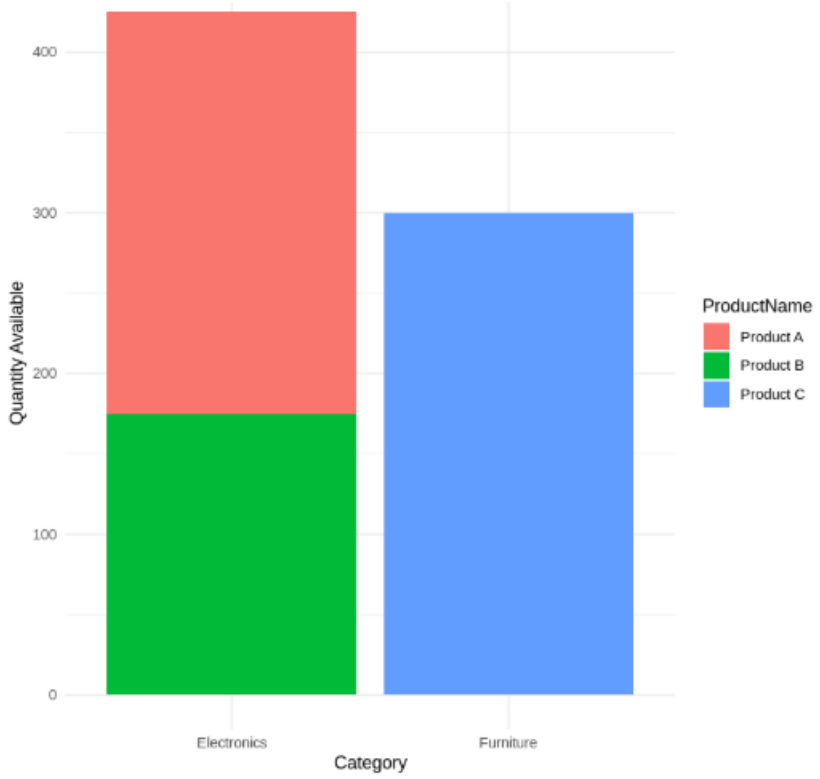
R Code

```
# Load library
library(ggplot2)
library(tidyr)
library(gridExtra)

# Dataset
inventory <- data.frame(
  ProductID=c(1,2,3),
  ProductName=c("Product A","Product B","Product C"),
  QuantityAvailable=c(250,175,300),
  Price=c(20,15,18),
  Category=c("Electronics","Electronics","Furniture")
)
```

```
# ----------------------------
# Bar Chart
# ----------------------------
ggplot(inventory,
       aes(x=ProductName,
           y=QuantityAvailable,
           fill=ProductName))+
geom_col()+
labs(title="Quantity Available for Each Product",
     x="Product Name",
     y="Quantity Available")+
theme_minimal()

# ----------------------------
# Stacked Bar Chart
# ----------------------------
ggplot(inventory,
       aes(x=Category,
           y=QuantityAvailable,
           fill=ProductName))+
geom_col()+
labs(title="Product Quantity by Category",
     x="Category",
     y="Quantity Available")+
theme_minimal()
```

```
# ----------------------------
# Inventory Table
# ----------------------------
grid.table(inventory)
```

output:



Result

The product inventory data was successfully analyzed using Python and R. A **bar chart** was created to visualize the quantity available for each product, a **stacked bar chart** was generated to compare quantities across product categories, and the **inventory table** displayed all product details. The visualizations provide an effective overview of inventory distribution and product availability.

Experiment 19: Survey Responses Analysis

Aim

To analyze survey response data using Python and R by creating a grouped bar chart, stacked bar chart, and survey response table for effective visualization of survey responses.

Procedure

1. Create the survey response dataset.
2. Load the required libraries.
3. Create a grouped bar chart to show the distribution of answers for Question 1.
4. Create a stacked bar chart to represent the overall distribution of responses for all three questions.
5. Display the survey response data in a table.
6. Analyze the charts and table.
7. Interpret the results.

Algorithm

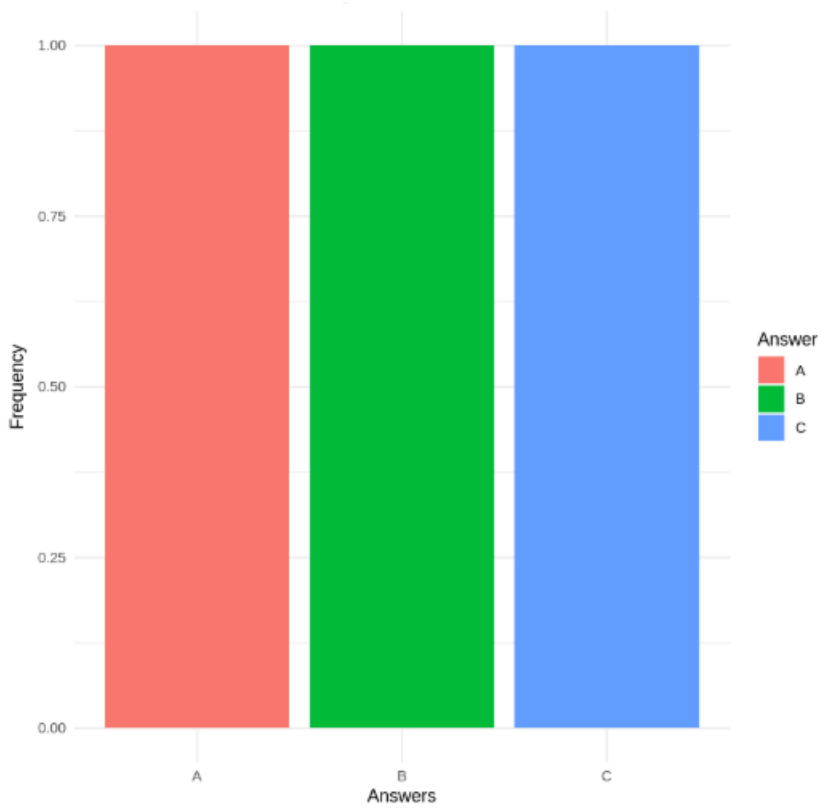
1. Start.
2. Create a dataset containing Survey ID, Question 1, Question 2, and Question 3 responses.
3. Read the dataset into the program.
4. Count the responses for Question 1 and create a grouped bar chart.
5. Convert the dataset into long format.
6. Create a stacked bar chart showing responses for all questions.
7. Display the survey response table.
8. Stop.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt

# Dataset
survey = pd.DataFrame({
    "Survey ID": [1, 2, 3],
    "Question 1": ["A", "B", "C"],
    "Question 2": ["B", "A", "A"],
    "Question 3": ["C", "D", "B"]
})

print(survey)

# ---------------------------------
# Grouped Bar Chart for Question 1
# ---------------------------------
q1 = survey["Question 1"].value_counts().sort_index()

plt.figure(figsize=(6,4))
plt.bar(q1.index, q1.values, color=["red","green","blue"])
plt.title("Distribution of Answers for Question 1")
plt.xlabel("Answers")
plt.ylabel("Frequency")
plt.show()
```

```
# --------------------------------
# Stacked Bar Chart
# --------------------------------
responses = pd.DataFrame({
    "Question 1": survey["Question 1"].value_counts(),
    "Question 2": survey["Question 2"].value_counts(),
    "Question 3": survey["Question 3"].value_counts()
}).fillna(0)

responses.T.plot(kind="bar", stacked=True, figsize=(7,5))
plt.title("Overall Distribution of Survey Responses")
plt.xlabel("Questions")
plt.ylabel("Count")
plt.show()
```

output:



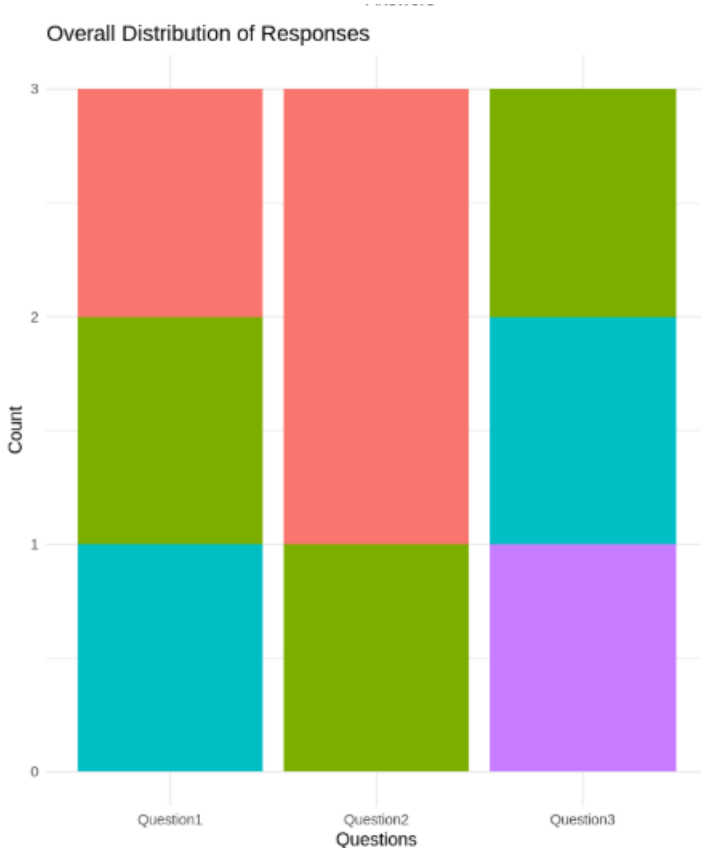
R Code

```
# Load Libraries
library(ggplot2)
library(tidyr)
library(gridExtra)


# Dataset
survey <- data.frame(
  SurveyID = c(1,2,3),
  Question1 = c("A","B","C"),
  Question2 = c("B","A","A"),
  Question3 = c("C","D","B")
)
```

```
# --------------------------------
# Grouped Bar Chart
# --------------------------------
ggplot(survey,
       aes(x = Question1,
           fill = Question1)) +
  geom_bar() +
  labs(title = "Distribution of Answers for Question 1",
       x = "Answers",
       y = "Frequency") +
  theme_minimal()

# --------------------------------
# Stacked Bar Chart
# --------------------------------
survey_long <- pivot_longer(
  survey,
  cols = c(Question1, Question2, Question3),
  names_to = "Question",
  values_to = "Response"
)

ggplot(survey_long,
       aes(x = Question,
           fill = Response)) +
  geom_bar() +
```

```
  labs(title = "Overall Distribution of Survey Responses",
       x = "Questions",
       y = "Count") +
  theme_minimal()


# ---------------------------------
# Survey Response Table
# ---------------------------------
grid.table(survey)
```

output:



Result

The survey response data was successfully analyzed using Python and R. A **grouped bar chart** was created to display the distribution of responses for **Question 1**, a **stacked bar chart** was generated to show the overall distribution of responses for all survey questions, and a **survey response table** displayed the

Experiment 20: Stock Analysis

Aim

To analyze stock price data using Python and R by creating a line chart, a bar chart showing the daily percentage change of Stock A, and a stock price table for effective visualization and analysis.

Procedure

1. Create the stock price dataset.
2. Load the required libraries.
3. Create a line chart to visualize the stock prices of Stock A, Stock B, and Stock C over the given dates.
4. Calculate the daily percentage change for Stock A.
5. Create a bar chart to display the daily percentage change.
6. Display the stock price data in a table.
7. Analyze the charts and interpret the results.

Algorithm

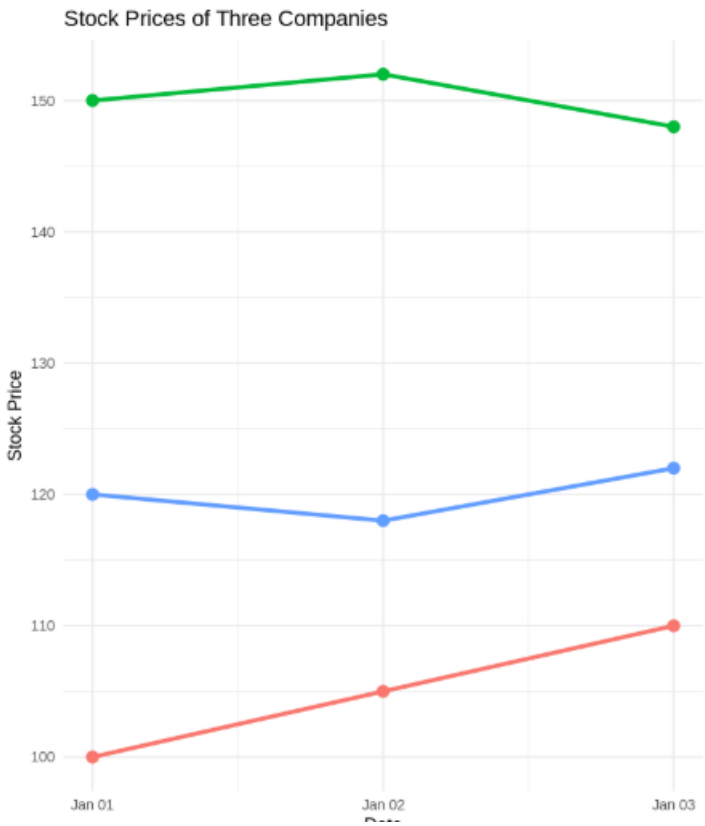
1. Start.
2. Create a dataset containing Date, Stock A, Stock B, and Stock C prices.
3. Read the dataset into the program.
4. Plot a line chart for all three stock prices.
5. Calculate the daily percentage change for Stock A.
6. Plot a bar chart for the percentage change.
7. Display the stock price table.
8. Stop.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt

# Create Dataset
stock = pd.DataFrame({
    "Date": ["2023-01-01", "2023-01-02", "2023-01-03"],
    "Stock A": [100, 105, 110],
    "Stock B": [150, 152, 148],
    "Stock C": [120, 118, 122]
})

# Convert Date column
stock["Date"] = pd.to_datetime(stock["Date"])

# Display Dataset
print(stock)

# ----------------------------
# Line Chart
# ----------------------------
plt.figure(figsize=(8,5))
plt.plot(stock["Date"], stock["Stock A"], marker='o', label="Stock A")
plt.plot(stock["Date"], stock["Stock B"], marker='o', label="Stock B")
plt.plot(stock["Date"], stock["Stock C"], marker='o', label="Stock C")

plt.title("Stock Prices of Three Companies")
```

```
plt.xlabel("Date")
plt.ylabel("Stock Price")
plt.legend()
plt.grid(True)
plt.show()

# ----------------------------
# Daily Percentage Change for Stock A
# ----------------------------
stock["Percentage Change"] = stock["Stock A"].pct_change() * 100

plt.figure(figsize=(6,4))
plt.bar(stock["Date"].dt.strftime("%Y-%m-%d")[1:],
        stock["Percentage Change"][1:],
        color="steelblue")

plt.title("Daily Percentage Change of Stock A")
plt.xlabel("Date")
plt.ylabel("Percentage Change (%)")
plt.show()
```

output:



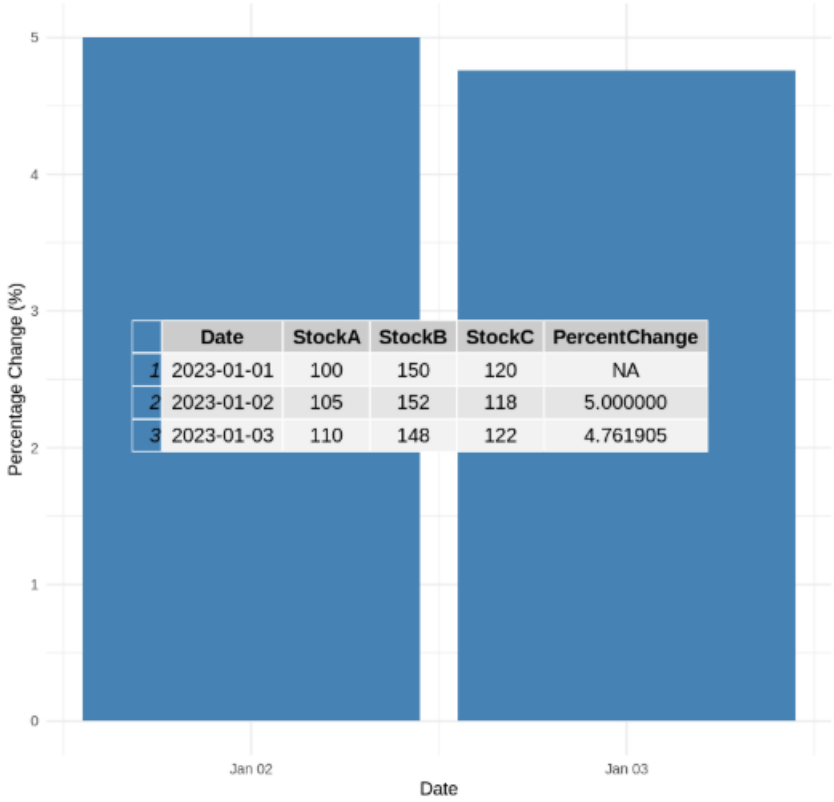
R Code

```
# Load Libraries
library(ggplot2)
library(tidyr)
library(gridExtra)


# Create Dataset
stock <- data.frame(
  Date = as.Date(c("2023-01-01",
                   "2023-01-02",
                   "2023-01-03")),
  StockA = c(100,105,110),
  StockB = c(150,152,148),
  StockC = c(120,118,122)
```

```
)

# ----------------------------
# Line Chart
# ----------------------------
stock_long <- pivot_longer(
  stock,
  cols = c(StockA, StockB, StockC),
  names_to = "Company",
  values_to = "Price"
)

ggplot(stock_long,
       aes(x = Date,
           y = Price,
           color = Company,
           group = Company)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(
    title = "Stock Prices of Three Companies",
    x = "Date",
    y = "Stock Price"
  ) +
  theme_minimal()
```

```
# ----------------------------
# Percentage Change for Stock A
# ----------------------------
stock$PercentageChange <- c(
  NA,
  (stock$StockA[2]-stock$StockA[1])/stock$StockA[1]*100,
  (stock$StockA[3]-stock$StockA[2])/stock$StockA[2]*100
)

change_data <- na.omit(stock)

ggplot(change_data,
       aes(x = as.factor(Date),
           y = PercentageChange)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Daily Percentage Change of Stock A",
    x = "Date",
    y = "Percentage Change (%)"
  ) +
  theme_minimal()

# ----------------------------
# Display Table
# ----------------------------
grid.table(stock)
```

output:



	Date	StockA	StockB	StockC	PercentChange
1	2023-01-01	100	150	120	NA
2	2023-01-02	105	152	118	5.000000
3	2023-01-03	110	148	122	4.761905

Result

The stock price data was successfully analyzed using Python and R. A **line chart** was created to visualize the stock prices of **Stock A, Stock B, and Stock C** over the given period. A **bar chart** was generated to show the **daily percentage change of Stock A**, and the **stock price table** displayed the stock prices along with the calculated percentage changes. The visualizations clearly illustrate stock price trends and daily performance, enabling effective analysis and comparison.